

DSA STUDY BLUEPRINT SERIES

# 110. Rotting Oranges Multi source BFS

## Leetcode 994

Simulating Rotting grids using Multi-source BFS

Section 10 • C++ Core Data Structures & Algorithms

# Core Concepts & Visual Blueprint

Theoretical models and memory architectures

---

## Key Principles

- **Problem:** Calculate time taken to rot all oranges.
- Push all initially rotten coordinates to a queue, then run BFS.

## Memory & Execution Blueprint

Rotten (2) → infects adjacent Fresh (1) ⇒ Rotting time increments.

# C++ Implementation Reference

Standard code layout and syntax

---

```
// BFS queue stores pair, int> (coords, time).  
// Process neighbors, decrementing fresh count and updating max time.
```

# Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

## Performance Table

| Aspect / Case    | Complexity |
|------------------|------------|
| Time Complexity  | $O(R * C)$ |
| Space Complexity | $O(R * C)$ |

## Key Practices & Gotchas

- **Important:** Fails if fresh oranges remain isolated from the grid components.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

# Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

## Execution Walkthrough

Let's trace Dijkstra distance relaxation updates on active vertices:

| Vertex popped | Adjacent edge checked | Current distance value | New path calculation | Relaxation action          |
|---------------|-----------------------|------------------------|----------------------|----------------------------|
| U (Dist = 2)  | U → V (Weight = 3)    | V (Dist = 7)           | New Dist = 2 + 3 = 5 | Relax: Update V (Dist = 5) |

# Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

---

## Key Variations & Follow-up Questions

- **Shortest Path choices:** BFS finds unweighted short paths; Dijkstra handles positive weights; Bellman-Ford resolves negative edges.
- **Spanning Trees:** Prim's builds MST from a start node; Kruskal's sorts all edges using disjoint sets.
- **Related LeetCode Problems:** #200 Number of Islands, #743 Network Delay Time, #787 Cheapest Flights Within K Stops.